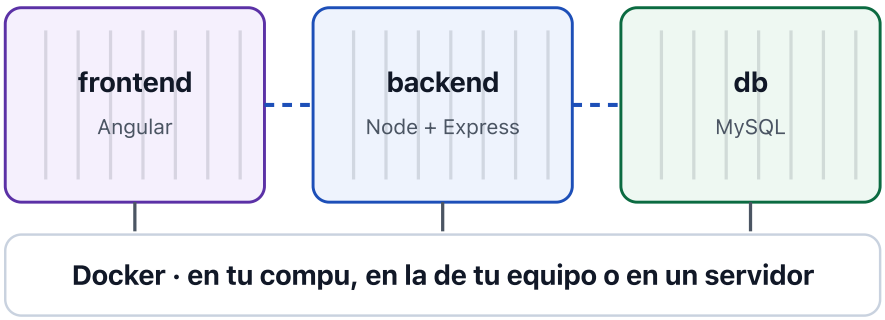
TÉCNICO UNIVERSITARIO EN PROGRAMACIÓN

Docker desde cero

Lo esencial para dockerizar tus proyectos



Guía de estudio

Qué es Docker, imágenes y contenedores, Dockerfile, Docker Compose, puertos, variables de entorno, datos de MySQL que no se pierden y cómo sumar Angular. Con los comandos y archivos listos para usar.

Versión práctica · octubre de 2026

Presentación interactiva: https://pedroibanez97.github.io/docker/
Repositorio con ejemplos y práctica: https://github.com/pedroibanez97/docker

Índice

Inicio

- La ruta de hoy
- Antes de empezar

Bloque 1. ¿Qué es Docker?

- “En mi compu funciona…”
- La idea de Docker
- Antes y después
- Preguntas rápidas

Bloque 2. Instalar y probar

- Instalá Docker
- ¿Funciona?
- Tu primer contenedor
- Si no funciona

Bloque 3. Imágenes y contenedores

- Imagen y contenedor
- Imágenes listas para usar
- Un servidor web con un comando
- Puertos: la puerta de entrada
- Ver qué está pasando
- Detener, iniciar y borrar
- Hacelo vos: tu servidor Apache

Bloque 4. Tu propia imagen: Dockerfile

- Tu API no está en Docker Hub
- La API de ejemplo: un Express común
- El Dockerfile, línea por línea
- .dockerignore: lo que no va en la imagen
- Construir la imagen
- Ejecutar tu API
- Cambié el código… ¿y no se ve?
- Hacelo vos: tu versión de la API

Bloque 5. Compose: tu API + MySQL

- Una app real tiene varias piezas
- El proyecto completo
- compose.yaml (1): la base de datos
- compose.yaml (2): tu API
- Variables de entorno: las claves van en .env
- Qué se sube a GitHub y qué no
- Levantar todo
- Probar la API: crear una tarea
- Dentro de Docker, localhost no es tu compu
- Los datos de MySQL: el volumen
- ¿Siguen los datos?
- Tus tablas: scripts SQL iniciales
- Ver la base con Workbench
- Los comandos de Compose
- Hacelo vos: API + MySQL

Bloque 6. Sumar Angular

- Angular en Docker: compilar y servir
- Angular llama a /api
- Los dos archivos del frontend
- Levantar la app completa

Bloque 7. Llevarlo a otra compu o a un servidor

- A la compu de otra persona
- A un servidor

Bloque 8. Si algo falla

- Primero, mirá
- Los errores más comunes

Bloque 9. Repaso

- Todo en un dibujo
- Machete de comandos
- Cuestionario rápido

Práctica final

- Paso a paso
- Adaptá tu código
- Explicá cómo se comunica tu proyecto
- Alternativa: el proyecto base
- Antes de entregar
- ¡Listo!

IDEA Inicio

La ruta de hoy

Explicar, mostrar y practicar. Al final, vas a dockerizar tu propio proyecto.

1 ¿Qué es Docker? — 15 min

2 Instalar y probar — 10 min

3 Imágenes y contenedores — 25 min

4 Tu imagen: Dockerfile — 25 min

5 Compose: API + MySQL — 40 min

6 Sumar Angular — 15 min

7 Llevarlo a otra compu — 10 min

8 Si algo falla — 10 min

9 Repaso — 10 min

★ **Práctica final** — Dockerizá tu proyecto

IDEA Inicio

Antes de empezar

Vos

- ✓ Docker instalado (lo hacemos en el bloque 2 si falta).
- ✓ VS Code y una terminal.
- ✓ Los ejemplos de la clase en tu compu:

```
Terminal
git clone
https://github.com/pedroibanez97/docker.g
it
```

Vamos a usar las carpetas `ejemplo` y `ejemplo-con-angular`.

Docente

Descargá las imágenes antes de clase (la red del aula puede ser lenta):

```
Terminal
docker pull hello-world
docker pull nginx:1.30-alpine
docker pull httpd:2.4-alpine
docker pull node:24-alpine
docker pull mysql:8.4
```

BLOQUE 1

¿Qué es Docker?

El problema que resuelve, en tres diapositivas.

Objetivo

Explicar con tus palabras para qué sirve Docker.

Tiempo

15 minutos · sin compu todavía

IDEA Bloque 1 · ¿Qué es Docker?

“En mi compu funciona…”

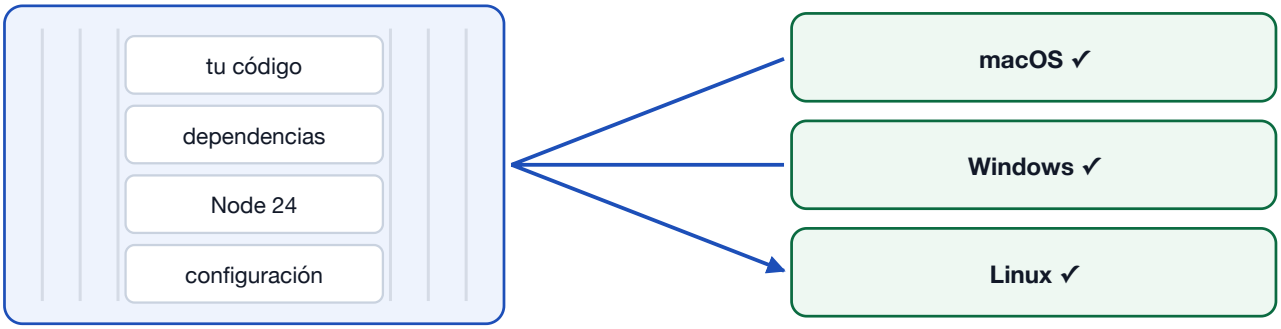


El código es el mismo. Lo que cambia es todo lo que hay alrededor: versiones, base de datos y configuración.

IDEA Bloque 1 · ¿Qué es Docker?

La idea de Docker

Docker mete tu app y todo lo que necesita en una caja que funciona igual en cualquier compu.



ANALOGÍA

Como un contenedor de barco: el barco no necesita saber qué hay adentro. Si es una caja estándar, la puede llevar.

IDEA Bloque 1 · ¿Qué es Docker?

Antes y después

Sin Docker: el README de siempre

1. Instalá Node (¿qué versión?)
2. Instalá MySQL
3. Creá la base y el usuario
4. Ejecutá el script de tablas
5. Armá el `.env`
6. `npm install`
7. `npm start`… y cruzá los dedos

Con Docker

Terminal

```
git clone
https://github.com/equipo/proyecto.git
cp .env.example .env
docker compose up -d --build
```

Lo único que hay que instalar es Docker.

REPASO Bloque 1 · ¿Qué es Docker?

Preguntas rápidas

PREGUNTA RÁPIDA

¿Docker reemplaza a Node, Express o MySQL?

No. Seguís usando lo mismo. Docker los ejecuta dentro de “cajas”.

PREGUNTA RÁPIDA

¿Docker arregla los bugs de mi código?

No. Hace que el entorno sea igual en todos lados. Los bugs viajan con el código.

PREGUNTA RÁPIDA

¿Es una máquina virtual?

No exactamente: es mucho más liviano y arranca en segundos. Para usarlo no hace falta saber más.

BLOQUE 2

Instalar y probar

Ideal: hacerlo antes de la clase. En clase, solo verificamos.

Objetivo

Tener Docker funcionando y ejecutar tu primer contenedor.

Tiempo

10 minutos (más la descarga)

IDEA Bloque 2 · Instalar y probar

Instalá Docker

macOS

1. Descargá Docker Desktop (https://docs.docker.com/desktop/setup/install/mac-install/): elegí **Apple silicon** (chip M1, M2…) o **Intel** (→ Acerca de esta Mac).
2. Arrastrá Docker a *Aplicaciones*.
3. Abrilo y aceptá el acuerdo.
4. Esperá a que diga *Engine running*.

Windows

1. En **PowerShell como administrador**: `wsl --install` y reiniciá.
2. Descargá e instalá Docker Desktop (https://docs.docker.com/desktop/setup/install/windows-install/) (dejá marcado *WSL 2*).
3. Abrilo desde Inicio y aceptá el acuerdo.
4. Esperá a que diga *Engine running*.

Requiere Windows 10 22H2 o 11, 64 bits, con virtualización activada.

Linux

1. Seguí la guía oficial de tu distribución (https://docs.docker.com/engine/install/) (en Ubuntu: “Install using the apt repository”).
2. Para usarlo sin `sudo`: `sudo usermod -aG docker $USER` y volvé a iniciar sesión.

> PARA SABER
>
> Docker Desktop es gratis para estudiar y para uso personal.

DEMO Bloque 2 · Instalar y probar

¿Funciona?

DÓNDE En cualquier terminal: Terminal (macOS), PowerShell (Windows) o la de VS Code.

Terminal

```
docker --version
docker compose version
```

QUÉ HACE Muestra la versión de Docker y de Docker Compose.

RESULTADO ESPERADO (LOS NÚMEROS PUEDEN VARIAR)

```
Docker version 28.5.1, build e180ab8
Docker Compose version v2.40.3-desktop.1
```

ERROR HABITUAL

`docker: command not found`: cerrá y abrí de nuevo la terminal. Si sigue, Docker no quedó instalado.

DEMO **Bloque 2 · Instalar y probar**

Tu primer contenedor

DÓNDE En cualquier terminal.

```
Terminal
docker run hello-world
```

QUÉ HACE Descarga una imagen de prueba y la ejecuta.

RESULTADO ESPERADO (RESUMIDO)

```
Unable to find image 'hello-world:latest'
locally
...
Hello from Docker!
This message shows that your installation
appears
to be working correctly.
```

¿Qué pasó?

1. Docker **descargó** la imagen `hello-world` (por eso dice *Unable to find… locally*: no es un error).
2. **Creó un contenedor** con esa imagen.
3. El contenedor **mostró el mensaje y terminó**.

IDEA **Bloque 2 · Instalar y probar**

Si no funciona

Ves esto…	Hacé esto
`Cannot connect to the Docker daemon`	Abrí Docker Desktop y esperá a que diga *Engine running*. En Linux: `sudo systemctl start docker`.
Windows: aviso sobre **WSL**	En PowerShell como administrador: `wsl --update` y reiniciá.
Windows: “Virtualization support not detected”	Activá la virtualización en la BIOS/UEFI (Intel VT-x o AMD-V).
Linux: `permission denied … docker.sock`	Usá `sudo` o agregate al grupo `docker` y volvé a iniciar sesión.

BLOQUE 3

Imágenes y contenedores

Las dos palabras que más vas a usar, y los comandos del día a día.

Objetivo

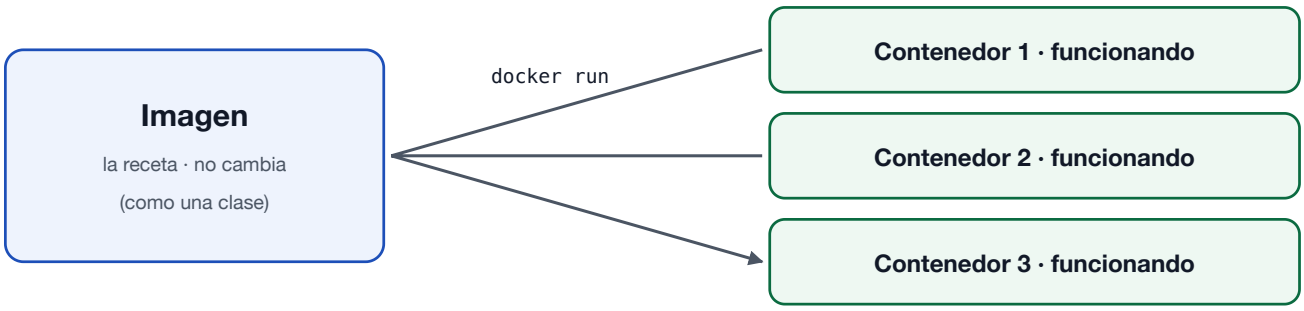
Levantar un servidor web en un contenedor, verlo en el navegador y manejarlo.

Tiempo

25 minutos · con la compu

IDEA Bloque 3 · Imágenes y contenedores

Imagen y contenedor



ANALOGÍA

Imagen = la receta. **Contenedor** = la torta hecha con esa receta. Con una receta hacés muchas tortas.

EN PROGRAMACIÓN

Como en programación: la imagen es la **clase** y cada contenedor es un **objeto** creado con `new`.

IDEA Bloque 3 · Imágenes y contenedores

Imágenes listas para usar

En Docker Hub (https://hub.docker.com) hay imágenes oficiales de casi todo. No hace falta instalar nada en tu compu.

`mysql:8.4`
MySQL 8.4, listo para arrancar.

`node:24-alpine`
Node 24 sobre un Linux mínimo.

`nginx:1.30-alpine`
Un servidor web.

Se escriben nombre:versión. Usá siempre la versión: `mysql:8.4`, no solo `mysql`.

DEMO Bloque 3 · Imágenes y contenedores

Un servidor web con un comando

DÓNDE En cualquier terminal.

Terminal

```
docker run -d --name web -p 8080:80 nginx:1.30-alpine
```

QUÉ HACE Crea y arranca un contenedor de nginx: `-d` en segundo plano, `--name web` le pone nombre, `-p 8080:80` lo conecta a tu compu.

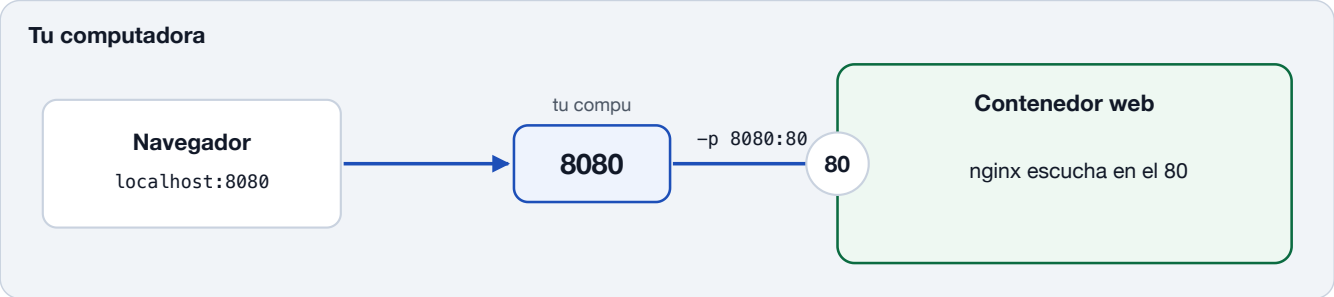
RESULTADO Abrí **http://localhost:8080**: aparece **“Welcome to nginx!”**.

ERROR HABITUAL

`port is already allocated` o `address already in use`: el 8080 de tu compu está ocupado. Usá otro número a la izquierda: `-p 8081:80`.

IDEA Bloque 3 · Imágenes y contenedores

Puertos: la puerta de entrada



`-p 8080:80` se lee: “el 8080 de mi compu va al 80 del contenedor”. Izquierda: tu compu. Derecha: el contenedor.

PREGUNTA RÁPIDA

¿Qué pasa si no ponés `-p`?

El contenedor funciona, pero desde tu compu no hay puerta para entrar.

DEMO Bloque 3 · Imágenes y contenedores

Ver qué está pasando

QUÉ HACE Lista los contenedores que están corriendo (`-a`: también los detenidos).

Terminal

```
docker ps
docker ps -a
```

RESULTADO (COLUMNAS PRINCIPALES)

```
IMAGE                STATUS         PORTS
NAMES
nginx:1.30-alpine    Up 2 minutes
0.0.0.0:8080->80/tcp   web
```

QUÉ HACE Muestra lo que imprime el contenedor (sus mensajes y errores).

Terminal

```
docker logs web
```

RESULTADO (DESPUÉS DE ABRIR LA PÁGINA)

```
... nginx/1.30.6
... "GET / HTTP/1.1" 200 896 ...
```

IDEA CLAVE

¿Algo no anda? **Primero mirá los logs.** Ahí casi siempre está la respuesta.

DEMO Bloque 3 · Imágenes y contenedores

Detener, iniciar y borrar

Comando	Qué hace
`docker stop web`	Lo detiene. El contenedor sigue existiendo (aparece en `docker ps -a`).
`docker start web`	Lo vuelve a arrancar, igual que estaba.
`docker rm web`	Lo borra (tiene que estar detenido).
`docker rm -f web`	Lo detiene y lo borra en un paso.

ATENCIÓN

Si **borrás** un contenedor, se pierde lo que tenía adentro (la imagen queda). Por eso los datos de MySQL van a ir en un **volumen** (bloque 5).

HACELO VOS Bloque 3 · Imágenes y contenedores

Hacelo vos: tu servidor Apache

10 minutos. Usá la imagen `httpd:2.4-alpine` (Apache escucha en el puerto 80 del contenedor).

1. Levantá un contenedor llamado `apache`, en segundo plano, en el puerto **8081** de tu compu.
2. Abrilo en el navegador. ¿Qué dice?
3. Mirá sus logs.
4. Detenelo, volvé a iniciarlo y comprobá que sigue andando.
5. Borralo (y borrá también `web`).

Solución

Terminal

```
docker run -d --name apache -p 8081:80
httpd:2.4-alpine
# navegador: http://localhost:8081  →
"It works!"
docker logs apache
docker stop apache
docker start apache
docker rm -f apache web
```

BLOQUE 4

Tu propia imagen: Dockerfile

Metemos una API de Express en una imagen.

Objetivo

Escribir un Dockerfile, construir la imagen y ejecutar tu API en un contenedor.

Material

Carpeta `ejemplo/backend`

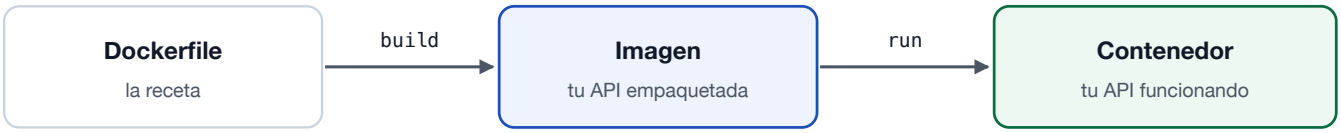
Tiempo

25 minutos

IDEA Bloque 4 · Tu propia imagen: Dockerfile

Tu API no está en Docker Hub

Para tu propio código, la imagen la creás vos con una receta: el Dockerfile.



IDEA Bloque 4 · Tu propia imagen: Dockerfile

La API de ejemplo: un Express común

```
ejemplo/backend/
├── Dockerfile        # la receta (nuevo)
├── .dockerignore     # qué no copiar (nuevo)
├── package.json
├── package-lock.json
└── src/
    └── index.js
```

IDEA CLAVE

El código no sabe nada de Docker. Solo lee la conexión a la base desde `process.env`, como cualquier proyecto con `.env`.

ejemplo/backend/src/index.js

```
import express from 'express';
import mysql from 'mysql2/promise';

const app = express();
app.use(express.json());

// Los datos de conexión llegan como variables de entorno (los pone Docker Compose)
const db = mysql.createPool({
  host: process.env.DB_HOST, // en Docker: "db", el nombre del servicio
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
});

app.get('/', (req, res) => {
  res.send('API funcionando');
});

app.get('/api/tareas', async (req, res) => {
  const [tareas] = await db.query('SELECT * FROM tareas');
  res.json(tareas);
});

app.post('/api/tareas', async (req, res) => {
  const titulo = req.body?.titulo;
  if (!titulo) return res.status(400).json({ error: 'Falta el título' });
  const [resultado] = await db.query('INSERT INTO tareas (titulo) VALUES (?)', [titulo]);
  res.status(201).json({ id: resultado.insertId, titulo });
});

// Si algo falla (por ejemplo, no hay conexión con MySQL), respondemos con el código del error
app.use((error, req, res, next) => {
  console.error(error);
  res.status(500).json({ error: error.code ?? error.message });
});

app.listen(3000, () => console.log('API escuchando en el puerto 3000'));

// Para que "docker stop" cierre la API al instante
process.on('SIGTERM', () => process.exit(0));
```

IDEA Bloque 4 · Tu propia imagen: Dockerfile

El Dockerfile, línea por línea

ejemplo/backend/Dockerfile

```
FROM node:24-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
CMD ["node", "src/index.js"]
```

1. `FROM`: partí de una imagen que ya tiene Node 24.
2. `WORKDIR`: trabajá en la carpeta `/app` (adentro del contenedor).
3. `COPY package*.json`: copiá la lista de dependencias.
4. `RUN npm ci`: instalá las dependencias.
5. `COPY . .`: copiá el resto del código.
6. `CMD`: al arrancar, ejecutá `node src/index.js`.

> PARA SABER
>
> El archivo se llama `Dockerfile`, sin extensión, y va en la carpeta del backend, junto a `package.json`.

IDEA Bloque 4 · Tu propia imagen: Dockerfile

`.dockerignore`: lo que no va en la imagen

.dockerignore

```
node_modules
.env
```

- ✓ `node_modules`: las dependencias se instalan adentro con `npm ci`.
- ✓ `.env`: tus contraseñas no tienen que quedar guardadas en la imagen.

> ANALOGÍA
>
> Es como el `.gitignore`, pero para Docker.

DEMO Bloque 4 · Tu propia imagen: Dockerfile

Construir la imagen

DÓNDE En la carpeta `ejemplo/backend` (donde está el Dockerfile).

Terminal

```
docker build -t mi-api .
```

QUÉ HACE Sigue la receta y crea una imagen llamada `mi-api`. El punto final significa “esta carpeta”.

RESULTADO ESPERADO (RESUMIDO)

```
 => [1/5] FROM docker.io/library/node:24-alpine
 => [2/5] WORKDIR /app
 => [3/5] COPY package*.json ./
 => [4/5] RUN npm ci
 => [5/5] COPY . .
 => => naming to docker.io/library/mi-api
```

ERROR HABITUAL

`failed to read dockerfile: open Dockerfile: no such file or directory`: estás en otra carpeta. Entrá con `cd ejemplo/backend`.

DEMO Bloque 4 · Tu propia imagen: Dockerfile

Ejecutar tu API

DÓNDE En cualquier terminal.

Terminal

```
docker run -d --name mi-api -p 3000:3000 mi-api
docker logs mi-api
```

QUÉ HACE Crea un contenedor con tu imagen y lo conecta al puerto 3000 de tu compu.

RESULTADO DE DOCKER LOGS

```
API escuchando en el puerto 3000
```

RESULTADO **http://localhost:3000** muestra **“API funcionando”**.

> PARA SABER
>
> `/api/tareas` todavía responde `{"error":"ECONNREFUSED"}`: no hay base de datos. La sumamos con Compose en el bloque 5.

IDEA Bloque 4 · Tu propia imagen: Dockerfile

Cambié el código… ¿y no se ve?

La imagen es una foto de tu código del momento en que la construiste. Si cambiás el código, hay que construirla de nuevo.

DÓNDE En `ejemplo/backend`.

Terminal

```
docker build -t mi-api .
docker rm -f mi-api
docker run -d --name mi-api -p 3000:3000 mi-api
```

QUÉ HACE Nueva imagen, borra el contenedor viejo y crea uno nuevo.

> PARA SABER
>
> Con Compose (bloque 5) esto se hace con un solo comando: `docker compose up -d --build`.

HACELO VOS Bloque 4 · Tu propia imagen: Dockerfile

Hacelo vos: tu versión de la API

10 minutos, en `ejemplo/backend`.

1. Construí la imagen y ejecutala en el puerto 3000.
2. En `src/index.js`, cambiá `'API funcionando'` por un mensaje tuyo.
3. Hacé que se vea el cambio en `http://localhost:3000`.
4. **Extra:** ejecutá otro contenedor de la misma imagen en el puerto **3001**. ¿Qué número cambia en `-p`?

Solución

Terminal

```
docker build -t mi-api .
docker rm -f mi-api
docker run -d --name mi-api -p 3000:3000
mi-api

# extra: cambia solo el número de la
izquierda
docker run -d --name mi-api-2 -p
3001:3000 mi-api
```

BLOQUE 5

Compose: tu API + MySQL

La API, la base y los datos, levantados juntos con un comando.

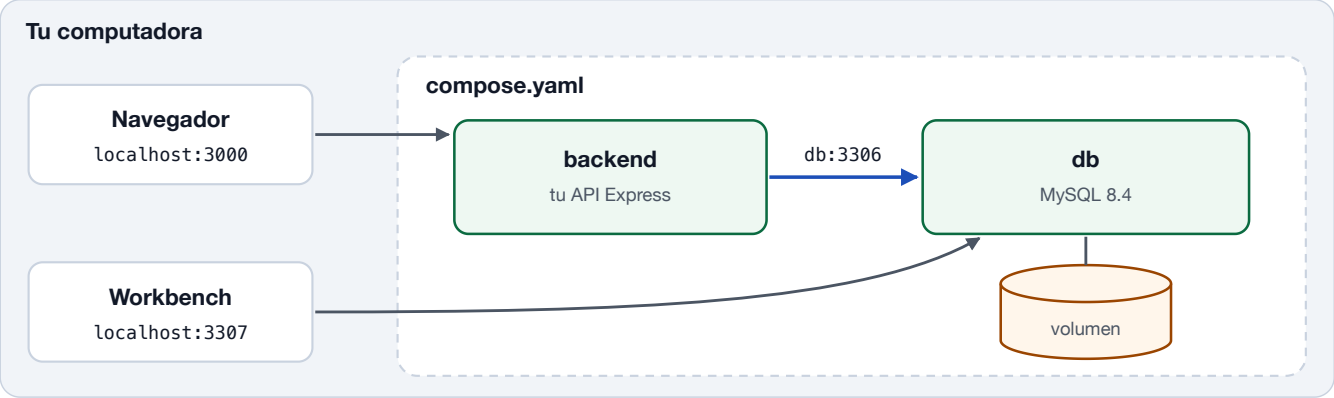
Objetivo
Levantar API + MySQL con Compose, usando variables de entorno y sin perder los datos.

Material
Carpeta `ejemplo`

Tiempo
40 minutos

IDEA Bloque 5 · Compose: tu API + MySQL

Una app real tiene varias piezas



Docker Compose describe todas las piezas en un archivo y las levanta con un comando.

IDEA Bloque 5 · Compose: tu API + MySQL

El proyecto completo

```
ejemplo/
├── compose.yaml      # las piezas y cómo se
conectan
├── .env.example      # plantilla de claves
├── backend/          # la API con su
Dockerfile
└── db/
    └── 01-tablas.sql # tablas y datos
iniciales
```

Es la misma estructura que vas a armar en tu proyecto:

- ✓ `compose.yaml` y `.env.example` en la raíz.
- ✓ Un `Dockerfile` en la carpeta del backend.
- ✓ Tus scripts `.sql` en una carpeta `db`.

IDEA Bloque 5 · Compose: tu API + MySQL

`compose.yaml` (1): la base de datos

yaml

```yaml
services:
  db:
    image: mysql:8.4
    environment:
      MYSQL_ROOT_PASSWORD:
${DB_ROOT_PASSWORD}
      MYSQL_DATABASE: ${DB_NAME}
      MYSQL_USER: ${DB_USER}
      MYSQL_PASSWORD: ${DB_PASSWORD}
    volumes:
      - datos-mysql:/var/lib/mysql        #
acá se guardan los datos (no se pierden)
      - ./db:/docker-entrypoint-initdb.d  #
scripts .sql que se ejecutan la primera vez
    ports:
      - "3307:3306"                       #
para verla con Workbench desde tu compu
    healthcheck:                          #
avisa cuando MySQL está listo
      test: ["CMD", "mysqladmin", "ping", "-
h", "127.0.0.1"]
      interval: 5s
      retries: 20
```

- `image`: usa la imagen oficial de MySQL.
- `environment`: crea la base y el usuario. Los valores salen del archivo `.env`.
- `volumes`: dónde guardar los datos y qué scripts ejecutar al crear la base.
- `ports`: `3307` para no chocar con un MySQL instalado en tu compu.
- `healthcheck`: bloque para copiar tal cual. Avisa cuando MySQL terminó de arrancar.

IDEA Bloque 5 · Compose: tu API + MySQL

`compose.yaml` (2): tu API

yaml

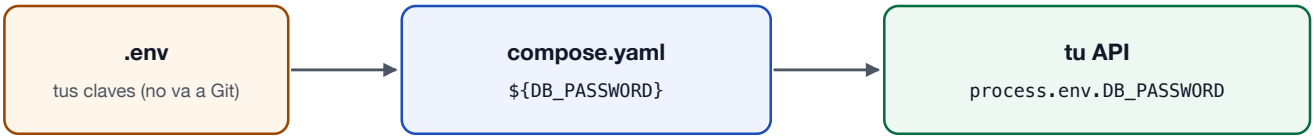
```yaml
  backend:
    build: ./backend
    ports:
      - "3000:3000"
    environment:
      DB_HOST: db                          #
el nombre del servicio, NO localhost
      DB_USER: ${DB_USER}
      DB_PASSWORD: ${DB_PASSWORD}
      DB_NAME: ${DB_NAME}
    depends_on:
      db:
        condition: service_healthy         #
espera a que MySQL esté listo

volumes:
  datos-mysql:
```

- `build`: construye la imagen con el Dockerfile de `./backend`.
- `ports`: la API en `localhost:3000`.
- `environment`: las variables que lee tu código con `process.env`.
- `DB_HOST: db`: la API encuentra a MySQL por el **nombre del servicio**.
- `depends_on`: no arranca hasta que MySQL esté listo.
- `volumes` (al final): crea el volumen de datos.

IDEA Bloque 5 · Compose: tu API + MySQL

Variables de entorno: las claves van en `.env`



ejemplo/.env.example

```
# Copiá este archivo como .env y cambiá las
claves.
# El archivo .env NO se sube a GitHub.
DB_ROOT_PASSWORD=cambiar-clave-root
DB_NAME=tareas_db
DB_USER=app
DB_PASSWORD=cambiar-clave-app
```

DÓNDE En la carpeta `ejemplo`.

Terminal

```
cp .env.example .env
```

QUÉ HACE Crea tu `.env` a partir de la plantilla. Después cambiá las claves.

IDEA Bloque 5 · Compose: tu API + MySQL

Qué se sube a GitHub y qué no

Archivo	¿A GitHub?
`Dockerfile`, `compose.yaml`	Sí
`.env.example` (claves de mentira)	Sí
`.env` (tus claves)	Nunca
`node_modules`	No

.gitignore (agregá estas líneas)

```
.env
node_modules
```

ATENCIÓN

Si subiste un `.env` por error, cambiá esas contraseñas. Borrar el archivo después no alcanza: queda en el historial.

DEMO Bloque 5 · Compose: tu API + MySQL

Levantar todo

DÓNDE En la carpeta `ejemplo` (donde está `compose.yaml`).

Terminal

```
docker compose up -d --build
docker compose ps
```

QUÉ HACE Construye la API, descarga MySQL y arranca todo en segundo plano. `ps` muestra cómo están.

RESULTADO DE DOCKER COMPOSE PS (COLUMNAS PRINCIPALES)

```
SERVICE   STATUS                   PORTS
backend   Up 3 seconds             0.0.0.0:3000->3000/tcp
db        Up 9 seconds (healthy)   0.0.0.0:3307->3306/tcp
```

RESULTADO `http://localhost:3000/api/tareas` devuelve las tareas guardadas en MySQL:

RESPUESTA

```
[{"id":1,"titulo":"Instalar Docker"},{"id":2,"titulo":"Levantar mi primer contenedor"}]
```

DEMO Bloque 5 · Compose: tu API + MySQL

Probar la API: crear una tarea

DÓNDE En cualquier terminal (o con Postman / Thunder Client: `POST` a `http://localhost:3000/api/tareas`).

macOS / Linux / Git Bash

Terminal

```
curl -X POST -H "Content-Type: application/json" -d '{"titulo":"Aprender Compose"}'
http://localhost:3000/api/tareas
```

Windows PowerShell

PowerShell

```
Invoke-RestMethod -Method Post -Uri http://localhost:3000/api/tareas -ContentType
'application/json' -Body '{"titulo":"Aprender Compose"}'
```

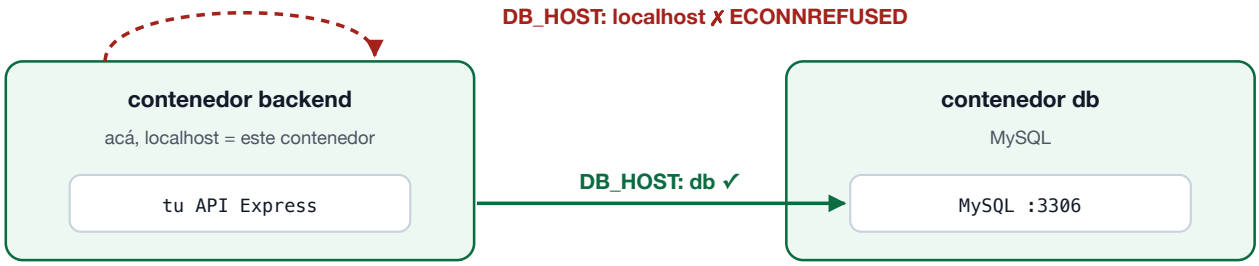
QUÉ HACE Envía una tarea nueva a la API, que la guarda en MySQL.

RESULTADO ESPERADO

```
{"id":3,"titulo":"Aprender Compose"}
```

IDEA Bloque 5 · Compose: tu API + MySQL

Dentro de Docker, `localhost` no es tu compu



Cada contenedor es su propia compu. La API encuentra a MySQL por el nombre del servicio: `db`.

ERROR HABITUAL

Con `DB_HOST: localhost`, `/api/tareas` responde `{"error":"ECONNREFUSED"}`. Es el error más común al dockerizar.

IDEA Bloque 5 · Compose: tu API + MySQL

Los datos de MySQL: el volumen



ANALOGÍA

El volumen es la **baulera** del edificio: aunque vacíes o cambies el departamento (el contenedor), lo que está en la baulera queda.

`docker compose down`	Los datos quedan
`docker compose down -v`	⚠ Borra los datos

DEMO Bloque 5 · Compose: tu API + MySQL

¿Siguen los datos?

DÓNDE En la carpeta `ejemplo`, después de crear la tarea “Aprender Compose”.

Terminal

```
docker compose down
docker compose up -d
```

QUÉ HACE Borra los contenedores y los vuelve a crear. El volumen no se toca.

RESULTADO En `/api/tareas` sigue estando “Aprender Compose”.

ATENCIÓN

`docker compose down -v` borra también el volumen: **se pierden todos los datos** y la base vuelve a empezar con el script inicial.

Usalo solo cuando quieras empezar de cero a propósito.

IDEA Bloque 5 · Compose: tu API + MySQL

Tus tablas: scripts SQL iniciales

ejemplo/db/01-tablas.sql

```
-- Se ejecuta solo la primera vez que se crea la base
CREATE TABLE tareas (
  id INT AUTO_INCREMENT PRIMARY KEY,
  titulo VARCHAR(200) NOT NULL
);

INSERT INTO tareas (titulo) VALUES ('Instalar Docker'), ('Levantar mi primer contenedor');
```

- Todo archivo `.sql` de la carpeta `db` se ejecuta cuando MySQL crea la base.
- **Solo la primera vez**: cuando el volumen está vacío.
- En tu proyecto, poné acá el script que ya usás para crear tus tablas.

ERROR HABITUAL

“Cambié el script y no pasa nada”: la base ya existía. En desarrollo, `docker compose down -v` y `up -d` la recrea (borrando los datos).

DEMO Bloque 5 · Compose: tu API + MySQL

Ver la base con Workbench

Nueva conexión (MySQL Workbench, DBeaver o VS Code)	
Host	`127.0.0.1`
Puerto	`3307`
Usuario	`app` (o el de tu `.env`)
Clave	la de `DB_PASSWORD` en tu `.env`
Base	`tareas_db`

RESULTADO ESPERADO

Ves la tabla `tareas` con los datos que cargaste desde la API.

ERROR HABITUAL

`Access denied` usando el puerto **3306**: te conectaste al MySQL instalado en tu compu. El de Docker está en el **3307**.

IDEA Bloque 5 · Compose: tu API + MySQL

Los comandos de Compose

DÓNDE SE EJECUTA

Siempre en la carpeta donde está `compose.yaml`.

Comando	Para qué
`docker compose up -d --build`	Construir y levantar todo (usalo también después de cambiar código)
`docker compose ps`	Ver cómo está cada servicio
`docker compose logs -f backend`	Ver los mensajes de la API en vivo (`Ctrl`+`C` para salir)
`docker compose stop` / `start`	Detener / volver a iniciar, sin borrar nada
`docker compose down`	Borrar los contenedores (los datos quedan)
`docker compose down -v`	⚠ **Borrar también los datos**

HACELO VOS Bloque 5 · Compose: tu API + MySQL

Hacelo vos: API + MySQL

15 minutos, en la carpeta `ejemplo`.

1. Creá el `.env` y levantá todo. Esperá a ver `db` como `(healthy)`.
2. Creá dos tareas y comprobá que aparecen en `/api/tareas`.
3. Conectate con Workbench y buscalas en la tabla.
4. Bajá todo con `down` y volvé a levantarlo. ¿Siguen?
5. Cambiá el mensaje de `/` en `backend/src/index.js` y aplicalo.

Solución

Terminal

```
cp .env.example .env
docker compose up -d --build
docker compose ps
# crear tareas con curl, PowerShell o Postman
docker compose down
docker compose up -d          # las tareas siguen
# después de cambiar el código:
docker compose up -d --build
```

BLOQUE 6

Sumar Angular

Un servicio más en el mismo `compose.yaml`.

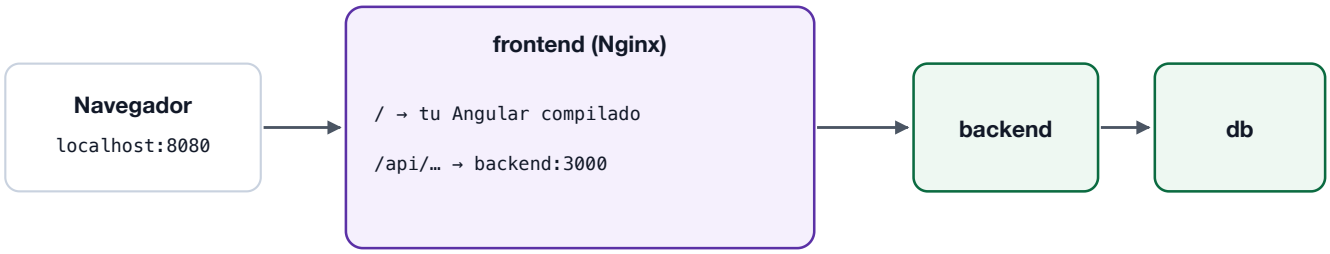
Objetivo
Entender cómo se suma el frontend y cómo llama a la API.

Material
Carpeta `ejemplo-con-angular`

Tiempo
15 minutos

IDEA Bloque 6 · Sumar Angular

Angular en Docker: compilar y servir



Docker hace el `ng build` y Nginx sirve los archivos. Nginx también pasa las llamadas a `/api` al backend.

ANALOGÍA

Una cocina y un mostrador: en la cocina (Node) se prepara la app; al mostrador (Nginx) solo llega el plato terminado.

IDEA Bloque 6 · Sumar Angular

Angular llama a `/api`

frontend/src/app/tareas.service.ts

```
  // URL relativa: el navegador se la pide a Nginx, y Nginx se la pasa al backend
  private apiUrl = '/api/tareas';
```

El código de Angular corre en el navegador, fuera de Docker. Por eso no puede usar `http://backend:3000`.

ERROR HABITUAL

`http://backend:3000/api` en Angular: el navegador no conoce ese nombre y la llamada falla.

RESULTADO ESPERADO

`/api/tareas`: el navegador se lo pide al mismo lugar de donde bajó Angular (Nginx), y Nginx se lo pasa al backend.

IDEA Bloque 6 · Sumar Angular

Los dos archivos del frontend

frontend/Dockerfile

```
# Paso 1: compilar Angular (como hacer "ng build")
FROM node:24-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# Paso 2: servir los archivos compilados con Nginx
FROM nginx:1.30-alpine
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=build /app/dist/frontend/browser /usr/share/nginx/html
```

frontend/nginx.conf

```
server {
  listen 80;
  root /usr/share/nginx/html;

  # Las llamadas a /api/... se pasan al backend ("backend" es el nombre del servicio)
  location /api/ {
    proxy_pass http://backend:3000;
  }

  # Todo lo demás es la aplicación Angular
  location / {
    try_files $uri $uri/ /index.html;
  }
}
```

ATENCIÓN

`dist/frontend/browser`: cambiá `frontend` por el nombre de tu proyecto Angular (lo ves al ejecutar `ng build`, en la línea *Output location*).

DEMO Bloque 6 · Sumar Angular

Levantar la app completa

compose.yaml (el servicio nuevo)

```
  frontend:
    build: ./frontend
    ports:
      - "8080:80"                          # la app completa: http://localhost:8080
    depends_on:
      - backend
```

DÓNDE En la carpeta `ejemplo-con-angular`.

Terminal

```
cp .env.example .env
docker compose up -d --build
```

RESULTADO **http://localhost:8080**: la app lista las tareas y permite agregar nuevas.



Resultado esperado

BLOQUE 7

Llevarlo a otra compu o a un servidor

Para esto sirve todo lo anterior.

Objetivo

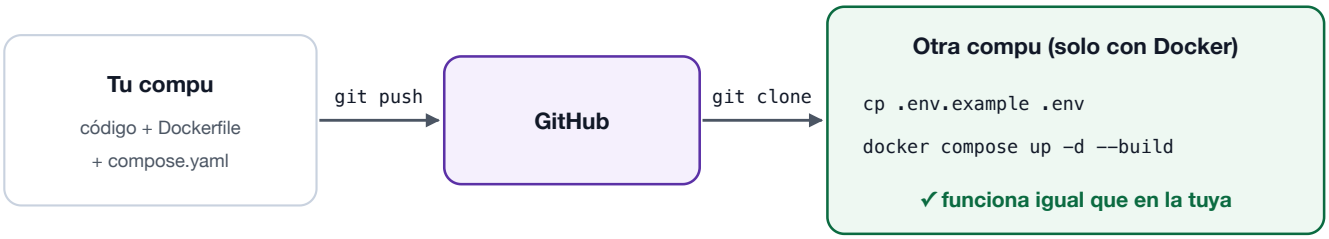
Ver por qué Docker facilita compartir y publicar un proyecto.

Tiempo

10 minutos

IDEA Bloque 7 · Llevarlo a otra compu o a un servidor

A la compu de otra persona



No hace falta instalar Node ni MySQL, ni crear la base a mano: todo está escrito en el proyecto.

IDEA Bloque 7 · Llevarlo a otra compu o a un servidor

A un servidor

Un servidor es otra compu (con Linux) que está siempre prendida. La idea es la misma.

1. En el servidor se instala Docker.
2. Se trae el proyecto: `git clone …`
3. Se crea el `.env` con **claves nuevas y fuertes** (nunca las de ejemplo).
4. `docker compose up -d --build`

PARA SABER

Para que la use otra gente faltan un dominio y HTTPS. Eso lo configura quien administra el servidor; no es parte de esta clase.

ATENCIÓN

En un servidor, sacá el `ports` de `db`: la base no tiene que quedar abierta a Internet. La API la sigue viendo por la red interna.

BLOQUE 8

Si algo falla

Los errores que más vas a ver y cómo salir de ellos.

Objetivo

Diagnosticar con dos comandos antes de cambiar nada.

Tiempo

10 minutos (y como referencia durante la práctica)

IDEA Bloque 8 · Si algo falla

Primero, mirá

DÓNDE En la carpeta de `compose.yaml`.

Terminal

```
docker compose ps -a
docker compose logs backend
```

QUÉ HACE `ps -a`: qué servicio está caído (`Exited`). `logs`: el mensaje de error exacto.

Leé el error antes de cambiar algo. Casi siempre dice qué pasa.

IDEA Bloque 8 · Si algo falla

Los errores más comunes

Ves esto…	Hacé esto
`Cannot connect to the Docker daemon`	Abrí Docker Desktop y esperá a que arranque.
`port is already allocated` / `address already in use`	Ese puerto de tu compu está ocupado: cambiá el número de la **izquierda** en `ports`.
`The "DB_PASSWORD" variable is not set` y `dependency failed to start`	Falta el `.env`: `cp .env.example .env`.
La API responde `{"error":"ECONNREFUSED"}`	La API usa `localhost` para la base: tiene que ser `DB_HOST: db`.
Cambié el código y no veo el cambio	`docker compose up -d --build`
Desaparecieron los datos	¿Usaste `down -v`? Sin `-v`, los datos quedan.
`Access denied for user` después de cambiar claves en `.env`	MySQL guarda las claves de la primera vez. En desarrollo: `docker compose down -v` y `up -d` (borra los datos).

BLOQUE 9

Repaso

Todo en un dibujo, un machete de comandos y un cuestionario rápido.

Objetivo

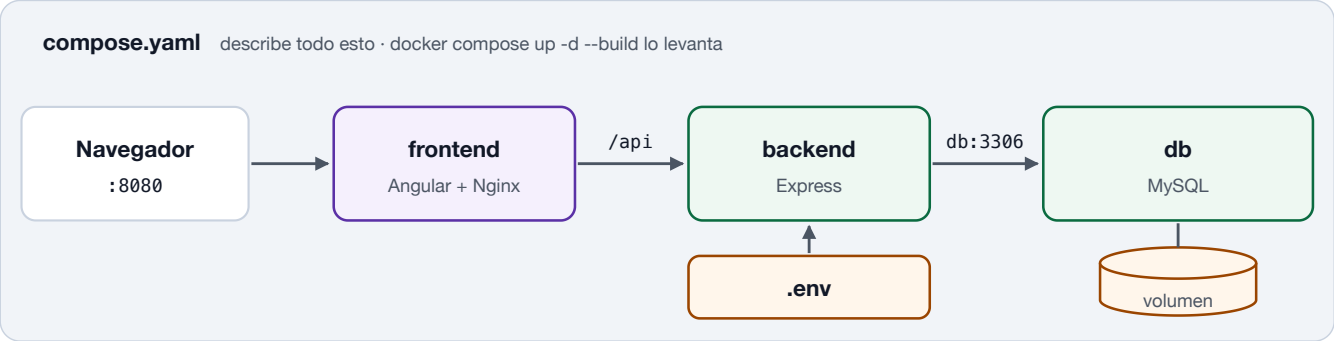
Confirmar que se entendió antes de la práctica.

Tiempo

10 minutos

REPASO Bloque 9 · Repaso

Todo en un dibujo



Imagen

La receta. La creás con un Dockerfile.

Contenedor

La app funcionando. Se puede borrar y recrear.

Puerto

`tu compu:contenedor`

Volumen

Donde quedan los datos de MySQL.

REPASO Bloque 9 · Repaso

Machete de comandos

Contenedores sueltos

Terminal

```
docker run -d --name web -p 8080:80
nginx:1.30-alpine
docker ps -a              # ver contenedores
docker logs web           # ver mensajes
docker stop web           # detener
docker start web          # volver a iniciar
docker rm -f web          # borrar
docker build -t mi-api .  # crear tu imagen
```

Con Compose (en la carpeta del proyecto)

Terminal

```
cp .env.example .env           # una sola vez
docker compose up -d --build   # levantar /
aplicar cambios
docker compose ps              # ver estado
docker compose logs -f backend # ver mensajes
docker compose stop            # detener
docker compose start           # volver a
iniciar
docker compose down            # borrar
contenedores (datos quedan)
```

REPASO Bloque 9 · Repaso

Cuestionario rápido

1. ¿Qué es un contenedor?

- La receta de la app
- **La app funcionando, creada a partir de una imagen ✓ correcta**
- Una carpeta de tu proyecto

La receta es la imagen. El contenedor es la app corriendo a partir de esa imagen.

2. En `-p 3001:3000`, ¿cuál es el puerto de tu compu?

- **3001 ✓ correcta**
- 3000

Izquierda: tu compu. Derecha: el contenedor.

3. En Compose, ¿qué host usa la API para conectarse a MySQL?

- `localhost`
- **`db`, el nombre del servicio ✓ correcta**

Dentro de un contenedor, `localhost` es el mismo contenedor.

4. Cambiaste el código de la API. ¿Qué ejecutás?

- `docker compose start`
- **`docker compose up -d --build` ✓ correcta**

La imagen tiene el código viejo: hay que construirla de nuevo.

5. ¿Qué comando borra los datos de MySQL?

- `docker compose down`
- **`docker compose down -v` ✓ correcta**

Sin `-v`, el volumen con los datos queda.

6. ¿Qué archivo nunca se sube a GitHub?

- `.env.example`
- **`.env` ✓ correcta**
- `compose.yaml`

`.env` tiene tus claves reales. `.env.example` es la plantilla sin secretos.

PRÁCTICA FINAL

Dockerizá tu proyecto

Que tu proyecto se levante con `docker compose up -d --build` en cualquier compu.

Tenés que poder

- Levantarlo con un comando
- Mostrar que funciona
- Explicar cómo se comunican sus partes

Con qué

Tu proyecto Express + MySQL (con o sin Angular). Si no podés usarlo, el proyecto base.

Tiempo

1 h 30 min a 3 h · individual o en parejas

HACELO VOS Práctica final

Paso a paso

1. Creá una rama: `git switch -c docker`
2. Copiá `Dockerfile` y `.dockerignore` de `ejemplo/backend` a tu backend.

IDEA CLAVE

No reescribas tu proyecto. Docker se agrega **alrededor**: tus rutas, controladores y servicios quedan como están.

3. Copiá `compose.yaml` y `.env.example` de `ejemplo` a la raíz de tu proyecto.
4. Poné tu script de tablas en `db/`.
5. Adaptá tu código (siguiente diapositiva).
6. `cp .env.example .env` y `docker compose up -d --build`

> **PARA SABER**
> Revisá en `compose.yaml`: la carpeta en `build` (por ejemplo `./backend` o `./server`) y el puerto en que escucha tu API.

HACELO VOS Práctica final

Adaptá tu código

1. La conexión a MySQL, con variables

Así queda en el ejemplo

```
const db = mysql.createPool({
  host: process.env.DB_HOST, // en Docker: "db", el nombre del servicio
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
});
```

Buscá en tu código `'localhost'`, `'root'` o tu contraseña escritos a mano, y reemplazalos por `process.env.…`.

2. El arranque

En el `CMD` del Dockerfile poné tu archivo de entrada (por ejemplo `src/app.js` o `index.js`). El puerto de `app.listen` tiene que ser el de la **derecha** en `ports`.

3. Angular (si llegás)

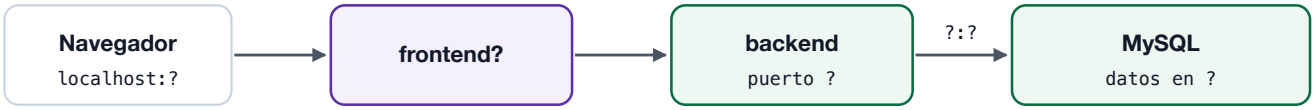
Copiá `Dockerfile` y `nginx.conf` de `ejemplo-con-angular/frontend` y el servicio `frontend` del compose. Usá `/api` como URL de la API.

> **PARA SABER**
> Si tu Angular usa `http://localhost:3000/api` y tu backend tiene CORS, también funciona en tu compu. Para un servidor, conviene `/api`.

HACELO VOS Práctica final

Explicá cómo se comunica tu proyecto

Dibujalo (en papel o en tu README) y respondé estas preguntas:



1. ¿Qué dirección abrís en el navegador?
2. ¿Cómo llega una petición al backend?
3. ¿Con qué host y puerto se conecta el backend a MySQL? ¿Por qué no `localhost`?
4. ¿Dónde se guardan los datos y qué pasa con ellos si hacés `docker compose down`?

HACELO VOS Práctica final

Alternativa: el proyecto base

Si no podés usar tu proyecto: una API chica de biblioteca (Express + MySQL), todavía sin Docker.

```
practica/proyecto-base/
├── package.json · package-lock.json
├── db/esquema.sql      # crea la base "biblioteca"
└── src/
    ├── index.js        # Express
    ├── db.js           # conexión (lee process.env)
    └── rutas/libros.js
```

Hacé los mismos pasos que con tu proyecto. Pistas:

- ✓ En `.env`: `DB_NAME=biblioteca` (el nombre que usa el script).
- ✓ El script está en `db/`: la misma carpeta del ejemplo.
- ✓ El backend es la carpeta misma: en el compose, `build: .`
- ✓ Probá `GET /api/libros`: tienen que aparecer 4 libros.

HACELO VOS Práctica final

Antes de entregar

- ✓ Se levanta con `docker compose up -d --build`.
- ✓ Los endpoints principales responden.
- ✓ Un dato creado sigue después de `docker compose down` y `up -d`.
- ✓ El `.env` **no** está en GitHub; `.env.example` sí.
- ✓ El README dice cómo levantarlo (los 3 comandos).
- ✓ Podés explicar el dibujo de la diapositiva anterior.

Qué se evalúa

- **Funciona:** levanta y responde.
- **Datos:** no se pierden al recrear.
- **Claves:** en `.env`, fuera de GitHub.
- **Explicación:** cómo se comunican las partes.

La prueba final: que otra persona clone tu repo y lo levante siguiendo solo tu README.

REPASO Práctica final

¡Listo!

Docker no cambia cómo programás. Cambia lo fácil que es que tu proyecto funcione en cualquier lado.

Guía de estudio

PDF con los conceptos, comandos y archivos.

Ejemplos

Repositorio (https://github.com/pedroibanez97/docker): carpetas `ejemplo` y `ejemplo-con-angular`.

Para profundizar

Versión completa de esta clase, con más detalle.